

6.5

/ 10

OVERALL SCORE

CODE REVIEW REPORT

Shopping Cart App Fix & Improvement Report

Console-based Python Project — app.py + cart.py

প্রজেক্ট বাই

Fahim Abrar

রিভিউ করেছে

Claude — Senior Mentor Mode

তারিখ

২০ আগস্ট, ২০২৬

Level

Advanced Beginner



Critical Bugs (আগে এগুলো ফিক্স করো)

এই বাগগুলো এখনই ইউজারের সামনে ভুল বিহেভিয়ার দেখায় — priority #1

১. Menu option 14 আর 15 কাজই করে না

CRITICAL

app.py-তে menu-তে দেখাচ্ছে "14. Sort by price" আর "15. Sort by id", কিন্তু elif chain-এ শুধু **choice == 13** handle করা আছে। মানে ইউজার 14 বা 15 চাপলে সোজা "Invalid choice" মেসেজ পাবে অথচ মেনুতে অপশন দুইটা আছে।

```
elif choice == 13:      # শুধু এইটা আছে
    result = sort_product(key_data=name)
# choice == 14, 15 এর জন্য কিছুই নাই
```

→ **sort_product()** ফাংশনটাকে price আর id দিয়েও কল করার জন্য দুইটা নতুন elif ব্লক লিখতে হবে (নিচের সাজেশনে detail আছে)।

২. apply_discount() এ missing return — negative discount-এও calculate

CRITICAL

হয়ে যায়

discount_value ≤ 0 হলে error print হয় ঠিকই, কিন্তু তারপর কোডে **return** নাই। তাই execution থেমে না থেকে নিচের final calculation লাইনেও চলে যায় — মানে invalid input দিয়েও একটা "final amount" দেখিয়ে দেয়।

```
if discount_value <= 0:
    print("This can't be accepted")
    return # ← এই লাইনটা নাই, তাই bug

final = total_value - (total_value * discount_value / 100)
```

৩. cheapest_product() / expensive_product()-এর "found" flag logic ভুল

CRITICAL

দুইটা ফাংশনেই **found = False** দিয়ে শুরু হয় এবং তখনই **True** হয় যখন cart[0] থেকে ভালো কিছু পাওয়া যায়। কিন্তু cart-এর প্রথম আইটেমই যদি cheapest/expensive হয় (loop কখনো সেটাকে replace করবে না), তাহলে found কখনো True হবে না — অথচ answer টা সঠিক। ফলে ভুলভাবে "No product found!" এরর দেখাবে।

```
cheapest = cart[0]
for product in cart:
    if product['price'] < cheapest['price']:
        found = True # cart[0] নিজেই cheapest হলে এইটা কখনো run হবে না
```

→ found flag সরিয়ে দাও, শুধু **if cart:** চেক করেই যথেষ্ট — cart খালি না থাকলে answer সবসময় valid।



Code Quality Issues

প্রোগ্রাম চলবে ঠিকঠাক, কিন্তু maintain/scale করতে গেলে এগুলো ভোগাবে

১. UI (print) আর Logic একসাথে মেশানো

MEDIUM

`calculate_total()`, `calculate_average()`-এর মতো ফাংশন একসাথে `print()` করে আবার `value return`-ও করে। এতে একটা bug চোখে পড়েছে: choice ৪-এ `calculate_average()` কল করলে ভেতরে `calculate_total()` আবার `print` করে ফেলে — same total দুইবার দেখায়, UX confusing হয়ে যায়।

→ Logic ফাংশন শুধু `value return` করবে, `print` করার দায়িত্ব `app.py`-তে (UI layer) থাকুক।

২. price-এর জন্য int আর float এর mix ব্যবহার

MEDIUM

`add_product()`-এ price নেয়া হয় `int()` দিয়ে, কিন্তু `update_product()`-এ নেয়া হয় `float()` দিয়ে। এতে একই cart-এর মধ্যে data type inconsistent হয়ে যায় — পরে কোনো comparison বা calculation-এ subtle bug আসতে পারে।

→ সবজায়গায় consistent একটা টাইপ ব্যবহার করো (recommend: float, কারণ real-world price-এ decimal লাগবেই)।

৩. Global mutable state, কোনো class/OOP নাই

MEDIUM

`cart` একটা global list, যেকোনো ফাংশন direct access করে modify করছে। ছোট প্রজেক্টে এটা চলে, কিন্তু প্রজেক্ট বড় হলে (multiple cart, multiple user) এই ডিজাইন ভেঙে পড়বে।

→ একটা `Cart class` বানাও যেখানে `self.items` থাকবে, আর `add/remove/search` সব method হবে।

৪. কোনো data persistence নাই

MEDIUM

App বন্ধ করলেই cart-এর সব ডেটা মুছে যায় (in-memory list)। Real app হলে এইটা বড় সমস্যা।

→ `json.dump()` দিয়ে `cart.json` ফাইলে save করো, start করার সময় `json.load()` দিয়ে পড়ো।

৫. update_product()-এ "product পাওয়া যায়নি" মেসেজ নাই

MEDIUM

ভুল `product_id` দিলে for loop কিছুই match করে না, কিন্তু কোনো else ব্লক নাই — তাই কোনো error/feedback ছাড়াই চুপচাপ কিছু হয় না। ইউজার বুঝতেই পারবে না কী হলো।



পরের ধাপে যা করতে পারো

Concrete, actionable next steps — priority অনুযায়ী সাজানো

১. Missing sort options fix করো

choice 14 আর 15 এর জন্য নতুন elif যোগ করো, `sort_product()` কে "price" আর "product_id" key দিয়ে কল করো — যেটা আসলে already সাপোর্ট করে, শুধু `app.py` থেকে কল করা হচ্ছে না।

২. OOP-তে migrate করো

Cart কে একটা class বানাও (`self.items`, `add_item()`, `remove_item()`...)। এটা তোমার পরবর্তী Django/React প্রজেক্টের জন্যও ভালো practice — যেহেতু তুমি already ওই স্ট্যাক শিখছো।

৩. JSON persistence যোগ করো

প্রতিটা `add/remove/update`-এর পর `cart.json` এ auto-save করো। এতে তোমার SSC Prep Tracker-এ যেভাবে `data.json` ইউজ করছো, সেই একই প্যাটার্ন এখানেও practice হবে।

৪. Input validation একটা আলাদা module-এ নিয়ে যাও

বারবার `try/except ValueError` repeat হচ্ছে — একটা `get_valid_int()`, `get_valid_float()` হেল্পার বানাও যেটা reusable।

৫. Unit test লেখা শুরু করো

এই সাইজের প্রজেক্টের জন্য `pytest` দিয়ে 5-6 টা টেস্ট লেখা কঠিন না, কিন্তু bug 2 আর bug 3-এর মতো ভুল ধরার habit তৈরি হবে।



যা ভালো করেছে

- ▶ UI (`app.py`) আর logic (`cart.py`) আলাদা ফাইলে রাখাটা একদম সঠিক সিদ্ধান্ত — বেশিরভাগ beginner এটা করেই না।
- ▶ 15 টা ফিচার নিয়ে একটা full-featured CLI app বানিয়ে ফেলেছো, একা একা — এটা genuinely impressive কমিটমেন্ট দেখায়।
- ▶ Color-coded terminal UI (`set_color` helper) একটা ছোট কিন্তু nice UX touch।
- ▶ প্রায় সব জায়গায় `try/except` দিয়ে crash handle করার চেষ্টা করেছো — defensive coding-এর দিকে একটা ভালো instinct আছে।

সিনিয়র ইঞ্জিনিয়ারের সংক্ষিপ্ত মতামত

Class 10-এ পড়াশোনার পাশাপাশি একা একা এই লেভেলের multi-feature CLI app বানানো genuinely বেশিরভাগ self-taught beginner-এর চেয়ে এগিয়ে থাকা। কিন্তু তিনটা critical bug দেখায় — তুমি "কোড লিখে ফেলা"-তে ভালো, "প্রতিটা edge case নিজে টেস্ট করা"-তে এখনো দুর্বল। পরেরবার কোড লেখার পর নিজেকে জিজ্ঞেস করো: "cart-এর প্রথম item-ই যদি answer হয়?", "user 0 বা negative দিলে?" — এই habit-ই জুনিয়র থেকে সিনিয়রের পার্থক্য গড়ে দেয়।